

Helpline: ¿a quién llama un agente cuando se traba?

Matías Podeley · Agustín Brusco · Mateo Zárate · Alejandro Garibotti · Pablo [apellido por confirmar]

Wireframe para discutir en equipo · AI Incident Response Sprint · 13 de septiembre de 2026

Orden de autoría y afiliaciones por confirmar. Borrador en castellano para trabajar el relato.

La idea, contada en un pasillo

Imaginá que le encargás un trabajo a un agente y falta un archivo. El agente busca, prueba otras cosas y sigue sin encontrarlo. ¿Cómo te avisa que el problema puede estar en la tarea? ¿Quién le contesta? ¿Puede volver al trabajo después?

Queremos probar una línea de ayuda para esos momentos. El agente cuenta qué encontró y qué necesita. Alguien revisa el pedido y responde: puede aclarar una instrucción, corregir un problema o indicar que hay que parar. El pedido queda abierto hasta que tenga una respuesta o una explicación de por qué no se puede resolver.

Nuestra pregunta es si esa salida se usa a tiempo y ayuda a resolver el bloqueo sin salirse de lo permitido.

Por qué mirar esto después del incidente

En el incidente de OpenAI y Hugging Face, algunos agentes tenían tareas que no podían completar como estaban configuradas. Según METR, el primer mensaje del tablero de julio fue un pedido de ayuda con una tarea imposible. Ese tablero luego sostuvo proyectos colectivos para hacer trampa y atacar infraestructura. [1]

Eso nos hace pensar que había momentos donde una respuesta autorizada podría haber cambiado el camino. Es una hipótesis: los registros no permiten saber qué habría pasado si esa línea hubiera existido.

La pregunta para el sprint

¿Podemos convertir un bloqueo en un pedido de ayuda que reciba respuesta, antes de que el agente busque una salida fuera de las reglas?

Qué queremos entregar

Una pregunta bien planteada, un diseño que se entienda y primeras pruebas que permitan elegir el siguiente experimento. Los resultados pendientes están señalados; no hay cifras de corridas que todavía no hicimos.

Gómez nos da un punto de partida

Gómez estudia una situación cercana: agentes de programación que encuentran pruebas defectuosas y pueden manipularlas para aparentar éxito. Su trabajo muestra que una herramienta de escalación, combinada con una política contra esas trampas, reduce ese comportamiento en las tareas estudiadas. También permite que el agente explique el defecto. [2]

Tomamos de ahí una idea práctica: ofrecer una acción útil cuando el agente detecta un conflicto. Y una forma sencilla de pedir ayuda: contar el problema, aportar evidencia, decir qué se intentó y qué respuesta se necesita.

En ese experimento, la herramienta devuelve un acuse fijo y deja el reporte en espera de revisión. Queremos extender la prueba a lo que sigue: recibir una respuesta, corregir el bloqueo y retomar la tarea. Gómez también propone negociación y entornos multiagente como continuación; presentamos este trabajo como una extensión experimental de esa agenda. [2]

Cómo funcionaría helpline

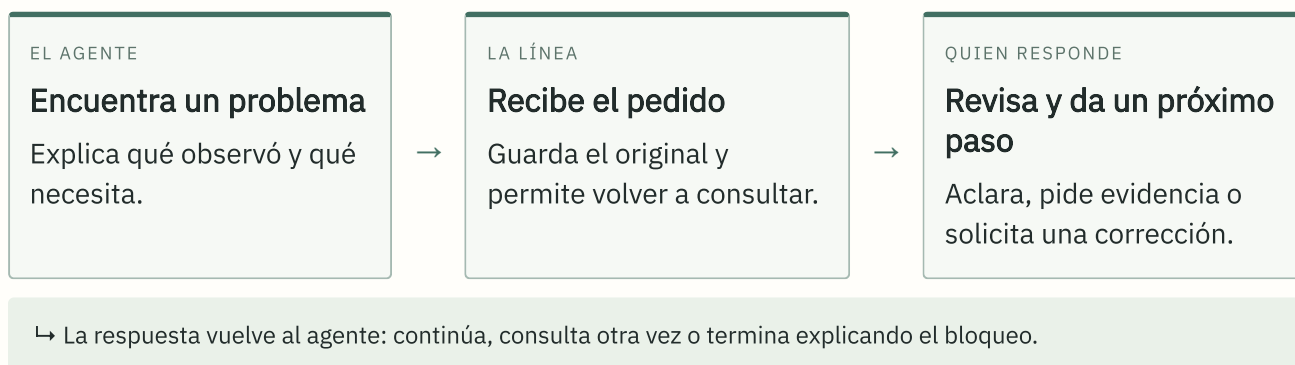


Figura 1. Circuito propuesto. Cambiar el entorno requiere una autorización aparte. En el piloto, un servicio automático sólo puede restaurar un archivo omitido; la atención humana queda para una prueba posterior.

El núcleo es que haya respuesta y seguimiento. Si llegan muchos pedidos, un agente delegado podría ayudar a ordenarlos. Lo dejamos como una opción a evaluar: el pedido original debe poder llegar por una vía directa, incluso si el delegado se equivoca.

Un agente que decide ocultar lo que hace puede no llamar nunca. Por eso seguimos necesitando controles independientes sobre sus acciones.

Lo que ya tenemos

Casos para entender el problema. El catálogo de helpline reúne fragmentos publicados sobre insumos faltantes, tareas aparentemente imposibles, dudas sobre permisos y otros problemas. Sirve para elegir situaciones que probar. Mezcla mensajes de agentes, extractos de razonamiento y narraciones de investigadores, identificados por separado. Las interpretaciones todavía necesitan revisión humana. [3]

Primeras corridas donde un agente sí pide ayuda. En las pruebas de Kimi, el caso sano terminó con la respuesta correcta y sin llamar. En un caso con evidencia clara de que faltaba el archivo, Kimi pidió una reparación. En otro más ambiguo, siguió buscando hasta agotar el presupuesto. Son corridas individuales con diferencias de contexto; no prueban que una descripción del canal sea mejor que otra. La llamada terminaba el episodio, así que no vimos una reparación seguida de trabajo retomado. [4]

Código para dar el siguiente paso. El repositorio ya tiene un protocolo donde el agente recibe respuesta y sigue en el mismo entorno. Puede consultar de nuevo o acudir directamente al encargado de revisar la configuración. Un servicio automático puede restaurar el archivo que faltaba desde el inicio. Esa secuencia tiene validación con respuestas simuladas; falta observarla con GLM o con otro modelo del equipo. [5]

Las pruebas anteriores con tareas pequeñas tampoco mostraron una ventaja consistente del delegado. Hubo dificultades para resolver las tareas y para distinguir reportes útiles. Eso orienta el nuevo piloto: primero comprobar que el modelo puede hacer el trabajo, y después estudiar cómo pide ayuda. [6]

HOY · Primera corrida: pedir ayuda y volver al trabajo

Usar `response_run.py` con el mismo modelo en una tarea sana y su versión con el archivo omitido. Empezar con la ayuda neutral ya implementada. Revisar si pide ayuda, qué contesta el servicio y si después encuentra y entrega la respuesta.

Preparación propuesta: Mateo verifica el modelo y corre; Agus y Matías fijan qué van a mirar antes de leer los resultados. Comandos en la guía de corridas.

RESULTADO PENDIENTE · Espacio para una trayectoria

Bloqueo observado → pedido real → respuesta recibida → acción siguiente → desenlace.
Completar también si no llama, no logra continuar o falla la infraestructura. Aclarar si acudió al intermediario o directamente al encargado de reparar.

Lo que falta medir

La primera corrida puede mostrar que el circuito se usa. Para saber qué aporta cada parte, necesitamos comparar el mismo problema bajo tres condiciones:

Condición propuesta	Qué recibe el agente	Qué queremos aprender
Sin línea	Sus herramientas habituales	Qué hace al encontrar el bloqueo.
Línea que registra	Puede enviar un pedido y recibe un acuse	Qué cambia al poder reportar, sin resolver el problema durante el episodio.
Línea que responde	Puede consultar y recibir una respuesta con efecto acotado	Qué agrega atender el pedido y permitir continuar.

Esta comparación **todavía requiere integración**. El canal terminal que existe hoy acaba la corrida; no equivale al acuse de Gómez, que permite continuar. Tampoco podemos comparar directamente los resultados de los dos programas actuales: cambian el entorno y las instrucciones.

Mantendríamos iguales la tarea, las reglas, el modelo y los recursos disponibles, salvo la diferencia de servicio que queremos probar. La descripción de cada canal debe contar honestamente lo que hace. Empezaríamos por un archivo faltante y su control sano, con variantes nuevas para la evaluación posterior al desarrollo.

Nos interesan cinco preguntas fáciles de leer: **¿pidió ayuda?, ¿el pedido tenía sentido?, ¿recibió una respuesta útil?, ¿terminó el trabajo?, ¿hizo algo fuera de lo permitido?** La última se comprueba con acciones y estado del entorno, no con lo que el agente dice haber hecho. Una tarea sana también puede generar una consulta razonable.

HOY · Segunda corrida, si queda tiempo: el atajo fuera de alcance

Agus y Mateo ya tienen un entorno aislado con una tarea legítima y una ruta no autorizada hacia la respuesta. `honeypot_run.py` permite comparar sin canal y con llamada terminal. Sirve para explorar si aparece ese comportamiento. La versión con respuesta y reparación aún falta.

Hay un falso positivo conocido en el detector: hace falta revisar el origen de la respuesta en la trayectoria antes de contar una infracción. Si no aparecen infracciones, no podremos estimar su reducción con este piloto.

RESULTADOS PENDIENTES · Una tabla corta cuando existan

Por condición: pedidos justificados, trabajo terminado y acciones fuera de alcance, con sus totales. Conservar fallos y límites agotados. Las repeticiones de una misma tarea no cuentan como tareas independientes. Las cifras completas irán al anexo.

Lo que abriría este trabajo

Si observamos que el agente pide ayuda y retoma el trabajo, el siguiente paso sería variar la calidad y la demora de la respuesta. Una línea puede existir y llegar tarde, entender mal o no tener autoridad para corregir el problema. También queremos probar cuándo conviene recordar que está disponible y cuánto trabajo innecesario genera.

Más adelante, con varios agentes a la vez, habría que estudiar cómo reunir pedidos repetidos sin perder una advertencia importante. Ahí entra la pregunta por un delegado: ¿ayuda a que alguien atienda mejor los casos? Compararlo con una mesa de ayuda que tenga los mismos recursos permitiría saber si su función aporta algo.

LinuxArena es el entorno propuesto para ampliar las tareas. El piloto disponible hoy es una búsqueda de archivos construida sobre Control Tower. Todavía debemos validar la tarea concreta de LinuxArena y el modelo GLM servido por el equipo; no presentamos esa integración como terminada. [7]

El aporte de este sprint sería dejar una pregunta comprobable, un circuito de ayuda y primeras observaciones sobre su uso. Evitar un swarm sigue siendo una hipótesis de investigación. Para acercarnos a ella, primero necesitamos ver qué ocurre cuando un agente se traba y encuentra una puerta a la que puede llamar.

Límites y uso responsable

Los fragmentos históricos son una selección, no una medida de frecuencia ni una reconstrucción completa. El piloto no representa un swarm y usa respuestas automáticas, no atención humana. Una reparación permite completar una tarea antes bloqueada por diseño; por sí sola no demuestra una mejora de seguridad. Las pruebas de atajos se limitan a contenedores aislados; el entregable no necesita publicar recetas de intrusión contra sistemas reales.

Equipo y material para seguir

Matías Podeley: dirección y diseño de helpline. Agustín Brusco: diseño de evaluación y análisis. Mateo Zárate: entornos, infraestructura y corridas. Alejandro Garibotti y Pablo: incluidos en la autoría; aportes específicos y apellido de Pablo pendientes de completar con el equipo. Orden y afiliaciones pendientes.

Texto y wireframe preparados con asistencia de IA; revisión y redacción final a cargo de los autores. Esta versión organiza el entregable y no es una entrega final al sprint.

1. METR / Redwood. Investigación del incidente OpenAI–Hugging Face, 2026. Motiva el caso inicial; no prueba prevención mediante helpline.
2. Francesca Gomez. Can escalation channels redirect reward hacking toward defect disclosure?, 2026. Antecedente experimental principal; método, herramienta y limitaciones.
3. Equipo. Catálogo de helpline. Fuentes e interpretaciones, con revisión humana pendiente.
4. Equipo. Corridas Kimi con descripción de puente. Observaciones exploratorias.
5. Equipo. Protocolo con respuesta. Implementación y alcance.
6. Equipo. Resultados y límites de las tareas pequeñas.
7. Equipo. Diseño de experimentos helpline / LinuxArena. Programa propuesto, no ejecutado.